COSC204
Computer Systems

Memory, Variables, and Pointers

Overview

- Last Lecture
  - C syntax
- This Lecture
  - Memory, Variables, Pointers
  - Source
    - K&R:
      - Brian Kernighan and Dennis Ritchie (1988), The C Programming Language. 2nd Edition. Prentice Hall. ISBN:0-13-110362-8
- Next Lecture
  - C Arrays, Malloc, and Free

Revision

- Memory is just a big array of bytes
- What is the meaning of a value in the array?
  - If we think of it as a character then it is so
  - If we think of it as a small integer then it is so
  - Can we think if it as an index into the array?

- memory[2] = 65 (integer)
- memory[2] = 65 (character ‘A’)
- memory[2] = 65 (and index into the array)

Value	Index
11	0
32	1
65	2
06	3
43	4
11	5
02	6
00	7

Pointers

- What is the *integer* value of
  - memory[3]
  - memory[memory[3]]
  - memory[memory[memory[3]]]

Value	Index
11	0
32	1
65	2
06	3
43	4
11	5
02	6
00	7

- This looks like it might be useful somehow…

Variables

- Let's give names (aliases) to some locations
  - (va, vb, and vc)
- va is at *location* 03
- va has the *value* 06
- Let's add some syntax
  - va is the same as memory[3]
  - *va is the same as memory[memory[3]]
  - **va is the same as memory[memory[memory[3]]]

Name	Value	Index
	11	0
	32	1
vc	65	2
va	06	3
	43	4
	11	5
vb	02	6
	00	7

Variables

- What is the *value* of vb?
- What is the *value* of *vb?
- What is the *value* of vc?

- From the table, what is the output:

```
vc++;
printf("%d, %d\n", *vb, **va);
```

- * is known as dereferencing
- ** is known as double-dereferencing

Name	Value	Index
	11	0
	32	1
vc	65	2
va	06	3
	43	4
	11	5
vb	02	6
	00	7

Names and Places

- Names are allocated to memory locations on declaration
  - This gives names to 2 (byte-sized) locations:

```
int main()
{
uint8_t a, b;
return 0;
}
```

- To find where in memory variable b is stored we go:

```
&b;     // take address of b (where, in the array, is b)
```

- Addresses might be bigger (or smaller) than an integer so a new type is needed. We call this new type a pointer

```
uint8_t *c = &a;// pointer holding the address of a
```

Addresses and Pointers

- What is the output of this program?

```
int main(void)
{
int vc;
int *vb, **va;

vc = 65;
vb = &vc;
va = &vb;     // why can't we say &&vc?
vc++;
printf("%d, %d\n", *vb, **va);
return 0;
}
```

Abstraction

- In the previous example we know the names of the locations (`va`, `vb`, and `vc`). We do not need to know the actual memory location that they occupy

- This is a form of abstraction from the hardware, most high-level programming languages give us this abstraction

- We no longer need to think about what is where, we rely on the compiler to abstract that for us

Why Pointers

- There are many reasons, including:
  - Abstraction
    - You don’t need to know which “object” to manipulate
  - The ability to change a parameter
    - Just like classes and references in Java
  - Multiple return values from a routine
    - Like multiple class parameters in Java
  - Efficiency
    - You don’t have to copy anything to pass a pointer

Call by Value

- In ***Call by Value*** the value of the variable is passed to the called routine
- As the routine has a ***copy*** of the parameter, changes do not propagate back to the caller
- Does Java do this?

```
void foo(char v)
{
v = 20;
}

int main(void)
{
char a;

a = 10;
printf("%d\n", a);
foo(a);
printf("%d\n", a);
return 0;
}
```

Call by Address

- In ***Call by Address*** the address of the parameter is passed to the routine and used there
- This is actually *call by value* of the *address,* (the location in that array we call memory)

```
void foo(char *v)
{
*v = 20;
}

int main(void)
{
char a;

a = 10;
printf("%d\n", a);
foo(&a);
printf("%d\n", a);
return 0;
}
```

Call by Address: Abstraction

- You can write a method that manipulates an object without knowing which object in advance

- In this case we don't need to know which long integers, we only need to know that it will be two long integers

```
long sum(long *a, long *b)
{
return *a + *b;
}
```

Call by Address: Change

- A method can change the value of a thing being pointed to
- This is like a Java object being passed to a Java method
  - Or using a wrapper class
- This is like references, but explicit

```
void swap(long *a, long *b)
{
long c;

c = *a;
*a = *b;
*b = c;
}
```

Call by Address: Multiple Results

- A C routine can return at most one value but can take multiple parameters
- If those parameters are result parameters (pointers) you can return one result but place other results where those pointers point

```
double mean_sd(long *values, long length, double *sd)
{
long *end, *current, sum;
double mean;

end = values + length;
sum = 0;
mean = *sd = 0;

for (current = values; current < end; current++)
        sum += *current;

mean = (double)sum / (double)length;

for (current = values; current < end; current++)
        *sd += (*current - mean) * (*current - mean);

*sd = sqrt(*sd / (double)length);

return mean;
}
```

Call by Address: Efficiency

- Only a copy of the address is put onto the stack
- Arrays are passed as the address of the start of the array

  `&array[0]`

```
char *strlower(char *string)
{
char *ch;

for (ch = string; *ch != '\0'; ch++)
   *ch = tolower(*ch);

return string;
}

int main(void)
{
char buffer[1024];

fgets(buffer, sizeof(buffer), stdin);
strlower(buffer);
puts(buffer);

return 0;
}
```

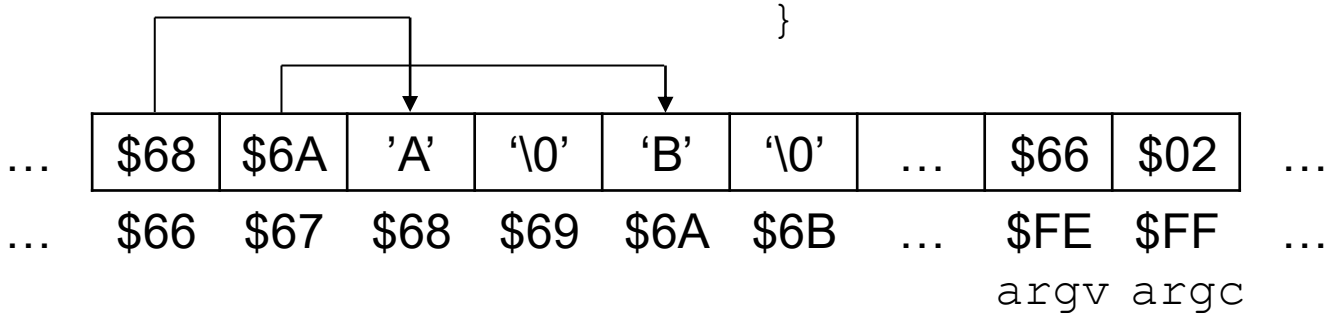
Call by Address: Arrays

- Variable length arrays:
  - Strings
    - Terminated by '\0'
  - argc, argv[],
    - Length specified
  - NULL terminated
    - Arrays of pointers

```
int main(int argc, char *argv[])
{
char **end;

end = argv + argc;
while (argv < end)
   puts(*argv++);

return 0;
}
```



Pointer Arithmetic

- ++ is used to move a pointer forwards in an array
- -- is used to move backwards
- These operators know about the ***type*** of the array
- ++ increments the address by `sizeof(*me)` bytes

```
double mean(long *values, long length)
{
long *end, *me, sum;

end = values + length;
sum = 0;

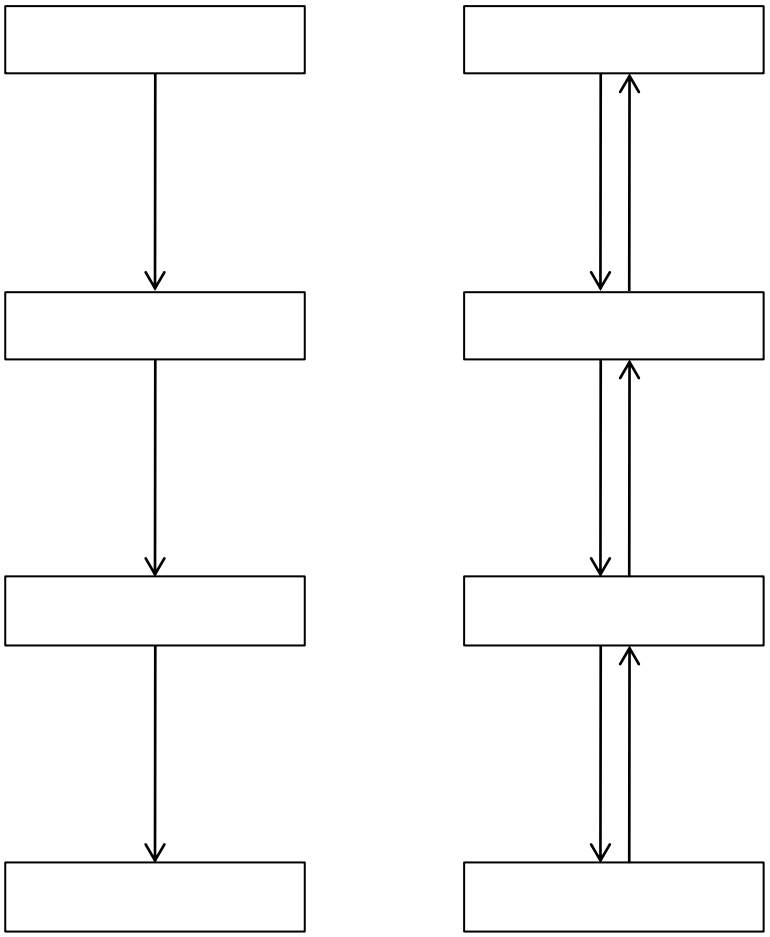
for (me = values; me < end; me++)
        sum += *me;

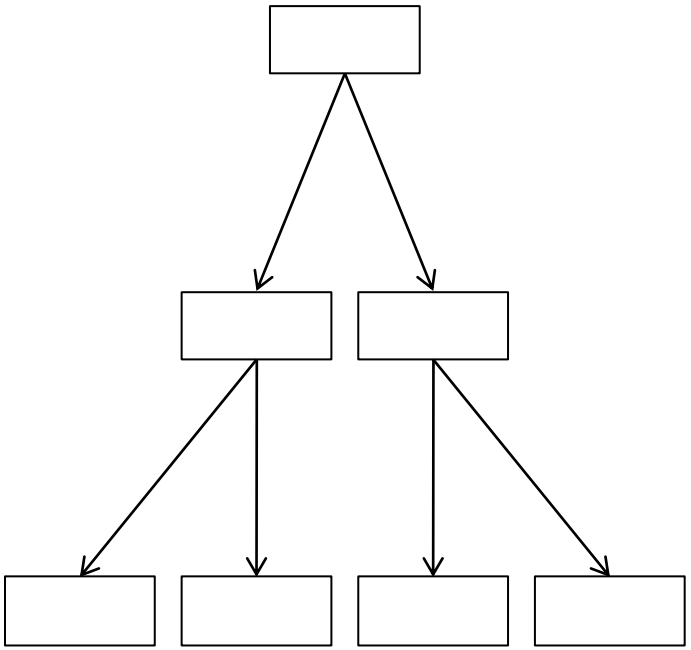
return (double)sum / (double)length;
}
```

Pointer Arithmetic

- Units are counted in elements being pointed to
  - This might include base-types, pointers, arrays, or structs
- Cannot do arithmetic outside one allocated block
  - With `int *b, a[10];` it is illegal to use `b = &a[0] + 50;`
  - But you can compute (but not use) `b = &a[0] + 10;`
  - This is (typically) not checked in C
- Cannot do arithmetic across blocks
  - But this is (typically) not checked either
- Operator precedence applies
  - What is the meaning of `*p++`

Dynamic Data Structures





What are these data structures?
Do they rely on pointers?

Homework

- What is the output of this program?
  - Think through the solution before typing it in to check

```
int method(int *x)
{
*x = 9;
}

int main(void)
{
int a = 3;

method(&a);
printf("%d\n", a);
return 0;
}
```